

HW5 report

112511071 楊文德



Top down code

```
int Memoized_Cut_Rod_Aux(const vector<int> &price, int rod_length, vector<int> &cut, vector<int> &first_cut)
{
    if (cut[rod_length] >= 0)
        return cut[rod_length];
    int q = INT_MIN;
    for (int i = 1; i <= rod_length; i++)
    {
        int temp = max(q, price[i] + Memoized_Cut_Rod_Aux(price, rod_length - i, cut, first_cut));
        if (temp > q)
        {
            q = temp;
            first_cut[rod_length] = i;
        }
    }
    cut[rod_length] = q;
    return q;
}

int Memoized_Cut_Rod(const vector<int> &price, int rod_length, vector<int> &first_cut)
{
    vector<int> cut(rod_length + 1, INT_MIN);
    cut[0] = 0;
    return Memoized_Cut_Rod_Aux(price, rod_length, cut, first_cut);
}
```

- 用途:
 1. Memorized_Cut_Rod: 初始化記憶陣列與初始條件
 2. Memorized_Cut_Rod_Aux: 遞迴計算最大收益，並透過記憶化避免重複計算
- 變數:
 1. Price[i]: 表示長度為 i 的棍子能賣多少錢
 2. Rod_length: 要切割的棍子總長度
 3. Cut[i]: 用來記錄這長度有沒有計算過
 4. First_cut: 用來記錄在某個長度下，最佳的第一刀應該切在哪裡
Ex : first_cut[5] = 2 即代表長度為 5 的棍子，第一刀切成 2+3



Bottom up code

```

int Bottom_up_Cut_Rod(const vector<int> &price, int rod_length, vector<int> &first_cut)
{
    vector<int> cut(rod_length + 1, 0);
    cut[0] = 0;
    for (int j = 1; j <= rod_length; j++)
    {
        You, 5 小時前 • HW5
        int q = INT_MIN;
        for (int i = 1; i <= j; i++)
        {
            if (q < price[i] + cut[j - i])
            {
                q = max(q, price[i] + cut[j - i]);
                first_cut[j] = i;
            }
        }
        cut[j] = q;
    }
    return cut[rod_length];
}

```

● 用途:

1. Bottom_up_Cut_Rod: 從小到大的方式計算每個長度的最佳收益，使用 DP(dynamic programming)的方式運行



時間比較

```

cout << "input rod of length: ";
cin >> rod_length;
vector<int> price(rod_length + 1, 0);
for (int i = 1; i <= rod_length; i++)
{
    price[i] = price[i-1] + rand()%10;
}

```

在這裡我透過 DP 的方式迭代逐步增加 price 值，這樣就能使用大數 rod_length 分析運算時間

```

input rod of length: 1000
-----
Top_down
Top_down Time : 0.003769 seconds

maximum price: 7000
number of pieces: 200

minimum price: 3000
number of pieces: 1000
-----
Bottom-up:
Bottom_up Time : 0.002523 seconds

maximum price: 7000
number of pieces: 200

minimum price: 3000
number of pieces: 1000

```

```

input rod of length: 10000
-----
Top_down
Top_down Time : 0.406616 seconds

maximum price: 76665
number of pieces: 3334

minimum price: 60000
number of pieces: 10000
-----
Bottom-up:
Bottom_up Time : 0.220775 seconds

maximum price: 76665
number of pieces: 3334

minimum price: 60000
number of pieces: 10000

```

我分別輸入 1000、10000 筆資料，然後我們可以觀察到 `top_down`、`bottom_up` 的時間差幾乎兩倍，明明時間複雜度分析下差不多，這是為什麼呢？

A：雖然 `top_down` 加上 `memorized` 的功能後能去除掉不必要的 `function call`，但無可避免的在呼叫函數時，在 `reference` 變數所帶來的時間增長

心得

這次的作業終於進入 `dynamic programming` 的環節了，以前高中時寫程式遇到 DP 問題都得兩手一攤，現在終於能進入系統性學習的環節，不過還是沒有那種 `divide and conquer` 的感覺...希望之後能夠掌握這項技能。